Bootcamp Project 4
Databricks Unity Catalog

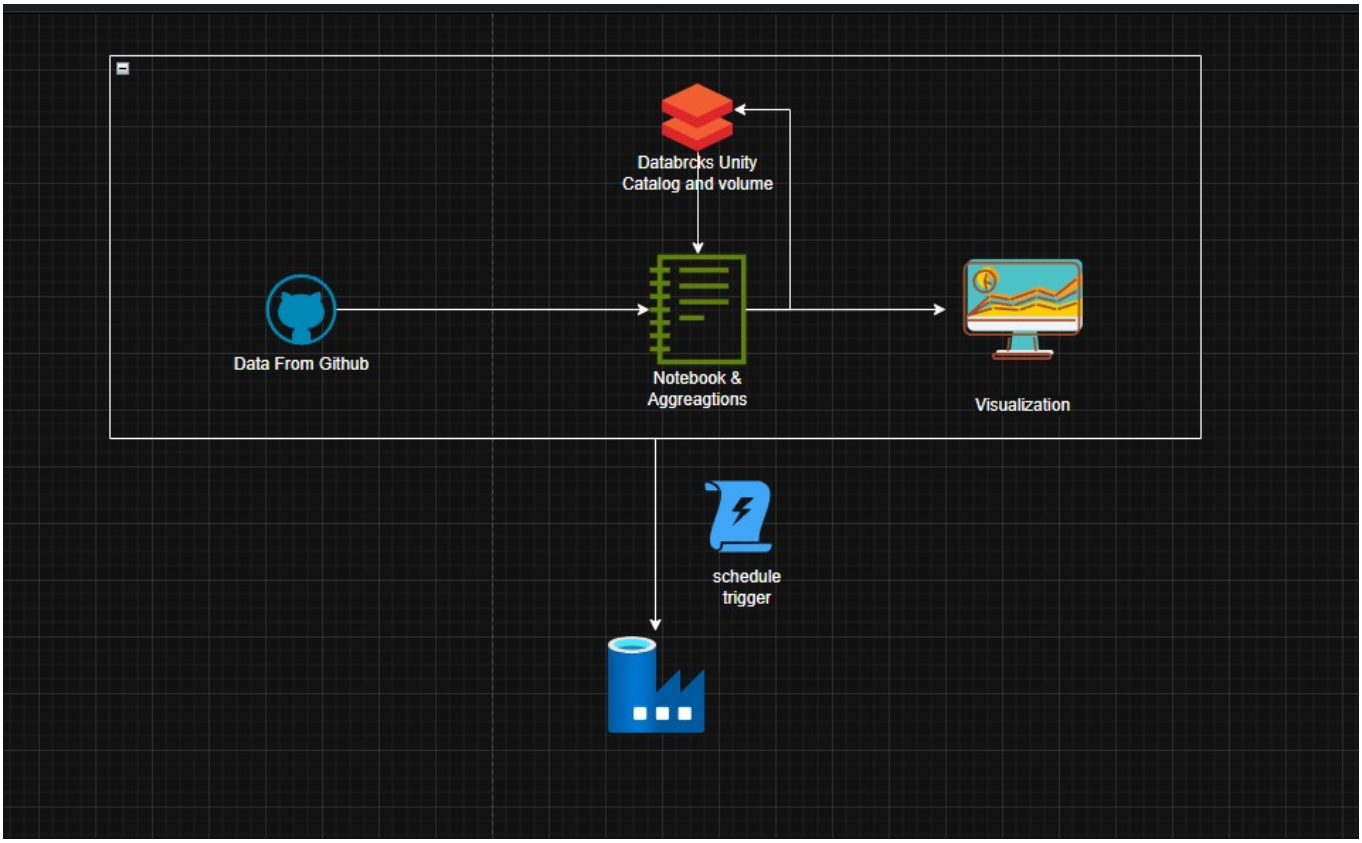
Project Overview

This project demonstrates an end-to-end **Modern Data Engineering pipeline** using:

- **Azure Data Factory (ADF)** → Orchestration & scheduling
- **Azure Databricks** → Data ingestion & transformation
- **Unity Catalog (Delta Tables / Volumes)** → Gold layer storage

The pipeline processes sales data from **2019, 2020, and 2021**, performs data cleaning and aggregation, and generates visual analytics using Plotly.

Architecture Overview



Data Source

Public GitHub CSV files:

- 2019.csv
- 2020.csv
- 2021.csv

Unity Catalog Setup

```
%sql
CREATE CATALOG project4_catalog
MANAGED LOCATION
'abfss://bronze@tarunadlsproject1.dfs.core.windows.net/project4_catalog/';
%sql
CREATE SCHEMA IF NOT EXISTS project4_catalog.data;
%sql
CREATE VOLUME IF NOT EXISTS project4_catalog.data.raw_data;
```

Define Schema:

```
schema = StructType([
    StructField("SalesOrderNumber", StringType(), True),
    StructField("SalesOrderLineNumber", IntegerType(), True),
    StructField("OrderDate", DateType(), True),
    StructField("CustomerName", StringType(), True),
    StructField("Email", StringType(), True),
    StructField("Item", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", FloatType(), True),
    StructField("Tax", FloatType(), True)
])
```

Ingesting Data:

```
import requests

urls = [
```

```
    "https://raw.githubusercontent.com/MicrosoftLearning/mslearn-
databricks/main/data/2019.csv",
    "https://raw.githubusercontent.com/MicrosoftLearning/mslearn-
databricks/main/data/2020.csv",
    "https://raw.githubusercontent.com/MicrosoftLearning/mslearn-
databricks/main/data/2021.csv"
]

workspace_path = "/Workspace/Users/tarunvinodh@gmail.com/rawdata"

dbutils.fs.mkdirs(workspace_path)

for url in urls:
    filename = url.split("/")[-1]
    response = requests.get(url)
    csv_content = response.content.decode('utf-8')
    dbutils.fs.put(f"{workspace_path}{filename}", csv_content,
overwrite=True)
```

```
df_2019 = spark.read.format("csv") \
    .schema(schema) \
    .load(f"{workspace_path}2019.csv")
df_2020=spark.read.format("csv").schema(schema).load(f"{workspace_path}2020.c
sv")
df_2021=spark.read.format("csv").schema(schema).load(f"{workspace_path}2021.c
sv")
display(df_2019)
display(df_2020)
display(df_2021)
```

Null Check:

```
df_2019 = df_2019.filter(
    (col("SalesOrderNumber").isNotNull()) &
    (col("SalesOrderLineNumber").isNotNull())
)
df_2020 = df_2020.filter(
    (col("SalesOrderNumber").isNotNull()) &
    (col("SalesOrderLineNumber").isNotNull()
```

```
)
df_2021 = df_2021.filter(
    (col("SalesOrderNumber").isNotNull()) &
    (col("SalesOrderLineNumber").isNotNull())
)
```

Deduplication:

```
from pyspark.sql.window import Window
w= Window.partitionBy("SalesOrderNumber",
"SalesOrderLineNumber").orderBy(col("OrderDate").desc())

df_2019 = df_2019.withColumn("row_num", row_number().over(w))
df_2019 = df_2019.filter(col("row_num") == 1).drop("row_num")

df_2020 = df_2020.withColumn("row_num", row_number().over(w))
df_2020 = df_2020.filter(col("row_num") == 1).drop("row_num")

df_2021 = df_2021.withColumn("row_num", row_number().over(w))
df_2021 = df_2021.filter(col("row_num") == 1).drop("row_num")

display(df_2019)
display(df_2020)
display(df_2021)
```

Aggregation:

```
df_2019_totalsales_per_customer= df_2019.groupBy("CustomerName") \
    .agg(
        sum(col("Quantity") * col("UnitPrice")).alias("TotalSales"),
        sum("Tax").alias("TotalTax")
    )

display(df_2019_totalsales_per_customer)

df_2020_totalsales_per_customer= df_2020.groupBy("CustomerName") \
    .agg(
        sum(col("Quantity") * col("UnitPrice")).alias("TotalSales"),
        sum("Tax").alias("TotalTax")
    )
```

```
display(df_2020_totalsales_per_customer)

df_2021_totalsales_per_customer= df_2021.groupBy("CustomerName") \
    .agg(
        sum(col("Quantity") * col("UnitPrice")).alias("TotalSales"),
        sum("Tax").alias("TotalTax")
    )

display(df_2021_totalsales_per_customer)
```

Saving as Delta Tables:

```
df_2019_totalsales_per_customer.write.format("delta") \
    .mode("overwrite") \

.save("dbfs:/Workspace/Users/tarunvinodh@gmail.com/project4_catlog/data/gold_
data/")

df_2020_totalsales_per_customer.write.format("delta") \
    .mode("overwrite") \

.save("dbfs:/Workspace/Users/tarunvinodh@gmail.com/project4_catlog/data/gold_
data/")

df_2021_totalsales_per_customer.write.format("delta") \
    .mode("overwrite") \

.save("dbfs:/Workspace/Users/tarunvinodh@gmail.com/project4_catlog/data/gold_
data/")
```

Visualization:

```
df_2019_with_year =
df_2019_totalsales_per_customer.withColumn("Year",lit(2019))
df_2020_with_year = df_2020_totalsales_per_customer.withColumn("Year",
lit(2020))
df_2021_with_year = df_2021_totalsales_per_customer.withColumn("Year",
lit(2021))
display(df_2019_with_year)
```

```
df_all_years =
df_2019_with_year.union(df_2020_with_year).union(df_2021_with_year)
```

```
#calculating total sales and tax per year
summary_per_year = df_all_years.groupBy("Year").sum("TotalSales", "TotalTax")
```

```
summary_pd = summary_per_year.toPandas()
summary_pd = summary_pd.sort_values("Year")
```

```
import plotly.express as px

fig = px.line(
    summary_pd,
    x="Year",
    y=["sum(TotalSales)", "sum(TotalTax)"],
    title="Total Sales & Total Tax Trend (2019-2021)",
    markers=True
)

fig.update_traces(mode="lines+markers")
fig.show()
```